



HANDOVER DOCUMENT

주간결 (Weekly-gyeol) 인수인계 문서

개인 시간관리 웹앱 — 무엇을 만들었고, 왜 그렇게 만들었으며,
지금 무엇이 위험한가.

대상 저장소 github.com/iphokorea1-boop/weekly-gyeol

운영 주소 weekly-gyeol.vercel.app

기준 커밋 20e1b7e (2026-08-19)

문서 작성 2026-08-19

0 이 문서를 읽는 법

이 문서는 기능 목록이 아니라 **판단의 기록**입니다. 코드가 무엇을 하는지는 코드를 읽으면 알 수 있지만, 왜 그렇게 되어 있는지는 읽어서 알 수 없기 때문입니다.

따라서 가장 중요한 장은 **3장(핵심 설계 결정)**입니다. 코드를 고치기 전에 그 장만은 반드시 읽으십시오. 거기 적힌 결정들은 대부분 실제로 무언가가 잘못된 뒤에 내려진 것이라, 근거를 모르고 되돌리면 같은 문제가 다시 발생합니다.

급한 사람을 위한 순서:

- **당장 배포·운행을 말아야 한다** → 6장, 7장
- **코드를 고쳐야 한다** → 3장, 4장, 그리고 별도 문서『초보 개발자를 위한 코드 안내서』
- **제품 방향을 판단해야 한다** → 1장, 9장

한 장 요약

항목	내용
무엇	할 일 · 반복 루틴 · 미배치 항목을 한 화면에서 관리하는 개인 시간관리 웹앱
누구	1차 사용자는 제작자 본인(고등학생). 다중 사용자 지원은 갖췄으나 아직 공개 배포 전
기술	Next.js 16.3.1 (App Router) · React 19 · TypeScript · Prisma 6 · PostgreSQL(Neon) · Tailwind CSS v4
배포	Vercel. <code>main</code> 브랜치 푸시 시 자동 배포
코드 규모	약 3,800줄 (설정·잠금파일 제외)
외부 인증 의존성	없음. 비밀번호 해싱은 Node 내장 <code>scrypt</code> , 세션은 자체 DB 테이블
가장 큰 미완성	비밀번호 재설정 없음 · 로그인 시도 제한 없음 · 개발용 DB 미분리 · 삭제 되돌리기 없음

1 제품: 무엇을 푸는가

1.1 문제 인식

학생용 계획 앱이 실패하는 지점은 대개 **의지가 아니라 과적(過積)**입니다. 하루에 12개를 적고, 4개를 하고, 죄책감을 느끼고, 앱을 지웁니다. 대부분의 도구는 20개를 적어도 아무 말을 하지 않으며, 오히려 많이 적을수록 성실한 것처럼 보여줍니다.

주간결은 반대 방향을 택했습니다. 하루가 얼마나 찼는지를 항상 보여주고 (`capacityPercent`), 연속 기록의 기준을 100%가 아니라 70%로 두어 한 번 못 지킨 날이 전체를 무너뜨리지 않게 했습니다. 이는 취향이 아니라 제품의 중심 가설이며, 바꾸려면 의식적으로 바뀌야 합니다.

1.2 세 가지 분류 — 이 앱의 핵심 개념

모든 항목은 단 하나의 **Task** 테이블에 저장되고, 두 개의 컬럼 값에 따라 세 종류로 해석됩니다. 별도 테이블이나 `type` 컬럼은 없습니다.

분류	판별 조건	의미
정기 루틴	<code>weekdays</code> 가 있음	요일 반복. 예: 월·수·금 운동. <code>dueDate</code> 는 무시됨
날짜 있는 할 일	<code>dueDate</code> 만 있음	특정 날짜의 일. 시간이 있으면 주간 격자에, 없으면 "종일" 행에
언젠가 할 일	둘 다 없음	미배치함. 날짜를 정하는 순간 자동으로 위 분류로 승격

왜 한 테이블인가

세 테이블로 나누면 "미배치 항목에 날짜를 붙이는" 동작이 **행 이동**이 되어, 트랜잭션·ID 변경·참조 갱신이 필요해집니다. 한 테이블에서는 `dueDate` 에 값을 하나 넣는 것으로 끝나고, 완료 기록(`TaskCompletion`)도 그대로 따라옵니다. 분류 로직은 `lib/task-utils.ts` 의 `taskKind()` 한 곳에만 있습니다.

1.3 화면 구성

경로	역할	내용
/	오늘	연속 기록 카드, 오늘 할 일(기한 지난 것 포함), 오늘의 루틴, 미배치함, 추가 폼
/week	주간 시간표	06-24시 7일 격자. 겹치는 일정은 나란히 배치. 현재 시각 선. 미니 달력과 미배치함 사이드바

/month	월간	달력 격자. 날짜 있는 할 일은 칩으로, 루틴 완료율은 칸 아래 얇은 막대로
/year	연간	12개월 히트맵(실천율), 배지 목록
/login	로그인·가입	유일한 비공개 아님 경로

2 개발 연혁

커밋 17개. 각 커밋이 무엇을 바꿨고 왜 필요했는지를 시간 순으로 정리합니다. 과거 결정의 맥락을 추적할 때 이 표에서 커밋 해시를 찾아 `git show` 로 보십시오.

커밋	제목	맥락
575f259	Initial commit from Create Next App	Next.js 기본 골격
ff7a10e	Add task/routine/floating data model, today & weekly views	세 분류 데이터 모델 확정, 오늘·주간 화면, Vercel 배포 준비
c697452	Add initial Postgres migration and Neon direct-url config	SQLite → PostgreSQL 전환. 마이그레이션에 필요한 <code>directUrl</code> 추가
4007c24	Run migrations outside the build	빌드 중 마이그레이션이 실패해 배포가 두 번 깨짐. 빌드에서 분리
e867117	Add month and year views, and an app shell	더 긴 기간 조망. 4개 탭 상단 내비게이션 추가
3d2b235	Rebuild the visual system on Radix Colors, shadcn/ui and Lucide	빛금 패턴이 칩 크기에서 뭉개짐. 아이콘+색 레일 방식으로 교체
7ff7765	Add streaks, XP and badges, and fix overdue completion	동기 부여 계층 추가. 이 과정에서 기한 지난 할 일은 체크가 저장되지 않던 버그 발견·수정
ea6557	Anchor all date logic to Korea Standard Time	서버(UTC)와 사용자(KST) 시차로 매일 0~9시에 "어제"로 표시되던 문제
6ab62f5	Make interactions respond on press instead of on response	체크박스가 서버 응답을 기다려 반응이 죽어 보임. 낙관적 UI 도입
5e76709	Give presses depth instead of only scale	누름 효과가 지나치게 미세해 체감되지 않음. 그림자 기반 깊이로 재설계. 동시에 격자 정렬 버그 2건 수정
e8155b7	Give the app accounts so it can have more than one user	계정·세션 도입. 동시에 IDOR 취약점 차단
20e1b7e	Let password managers recognise the sign-in form	아이디 칸의 <code>autocomplete</code> 토큰 교정

753c3d9	Document the project for handover and for a first-time reader	이 문서와 코드 안내서. 작성 중 <code>.env.example</code> 에 <code>DATABASE_URL_UNPOOLED</code> 가 빠져 있어 첫 마이그레이션에서 막히는 문제를 발견. 수정
efb6251	Show Korean public holidays on every calendar view	공휴일이 화면에 전혀 없어 쉬는 날 위에 일정을 잡게 됨. 저장하지 않고 계산하는 방식으로 추가(3.9)
408c373	Let tasks be dragged around the calendar	일정을 옮기려면 지우고 다시 만들어야 했고 그때 완료 기록이 사라짐. 포인터 기반 드래그 도입(3.10)
925acef	Email an account that has gone quiet for a few days	며칠 활동이 없으면 Vercel 크론이 메일로 알림. 같은 날 개발용 DB를 분리했고, 그 계기는 8.2의 사고
123d753	Remove a seed script that would have wiped the live database	중간 점검. <code>db:seed</code> 가 필터 없이 전체를 지우고 있었음. 그 외 미사용 코드. 자산 정리와 <code>README.md</code> 재작성

커밋 메시지를 읽으십시오

이 저장소의 커밋 메시지는 "무엇을 바꿨다"가 아니라 "왜 그것이 문제였고, 어떻게 확인했는가"를 담고 있습니다. 특정 코드가 왜 그 모양인지 궁금하면 `git log -p --follow <파일>` 이 이 문서보다 정확합니다.

3 핵심 설계 결정

여기 있는 결정 대부분은 무언가가 실제로 잘못된 뒤에 내려졌습니다. 근거를 모른 채 되돌리면 같은 문제가 재발합니다.

3.1 모든 날짜는 한국시간 기준, UTC 자정으로 저장

문제. Vercel 서버는 UTC로 동작하고 사용자는 KST입니다. 코드가 `date.getFullYear()` 같은 *서버 지역시간* 함수를 쓰고 있었기 때문에, 매일 **0시부터 9시 사이에는 앱이 아직 어제라고 판단했습니다**. 연속 기록이 날짜 단위로 계산되므로 밤에 목표를 채워도 기록이 어긋났습니다.

결정. 서버 시간대를 KST로 고정하는 대신, **날짜 계산 자체를 시간대와 무관하게** 다시 썼습니다. 한국은 1988년 이후 서머타임이 없어 +9 고정 오프셋이 정확합니다.

lib/task-utils.ts

```
export const KST_OFFSET_MINUTES = 9 * 60;

/** UTC 게터로 서울 벽시계 값을 읽도록 순간을 이동시킨다. */
function seoulClock(date: Date): Date {
  return new Date(date.getTime() + KST_OFFSET_MINUTES * 60_000);
}

/** 표준 날짜 값 = 그 순간이 속한 서울 달력 날짜의 UTC 자정.
  저장된 값에 다시 적용해도 결과가 같다(먹등). */
export function toDateOnly(date: Date): Date {
  const k = seoulClock(date);
  return new Date(Date.UTC(k.getUTCFullYear(), k.getUTCMonth(), k.getUTCDate()));
}
```

왜 서버 TZ 고정이 아니라 이 방식인가

`TZ=Asia/Seoul` 은 증상만 가립니다. Vercel은 UTC지만 개발자의 로컬 컴퓨터는 KST라, 같은 코드가 두 환경에서 다르게 동작하는 문제가 그대로 남습니다. 실제로 노션에서 일정을 가져올 때 날짜가 하루씩 밀린 사고가 정확히 이 원인이었습니다. 지금 방식은 **어디서 실행하든 결과가 동일합니다**.

건드릴 때 주의

`lib/task-utils.ts` 안의 날짜 함수는 전부 **UTC 게터** (`getUTCDate()` 등)로 작성돼 있습니다. 여기에 `getDate()`, `new Date(y, m, d)`, `toLocaleDateString()` 을 시간대 지정 없이 섞으면 조용히 하루가 밀립니다. 화면 표기는 반드시 `formatKo()` 를 쓰십시오 (내부에서 `timeZone: "UTC"` 를 강제합니다).

3.2 연속 기록의 기준은 100%가 아니라 70%

결정. 하루 예정 항목의 70% 이상을 끝내면 그날은 지킨 날입니다. 일정이 없는 날은 기록을 끊지 않고 중립으로 넘어가며, **오늘은 목표를 채우기 전까지 실패로 계산되지 않습니다.**

lib/gamification.ts

```
export const DAILY_GOAL_RATE = 0.7;

// 오늘은 실제로 채워지기 전까지 중립 - 아침의 미완료가
// 연속 기록이 끊긴 것처럼 보이면 안 된다.
let cursor = dayStatus(tasks, today).outcome === "kept"
  ? today
  : addDays(today, -1);
```

근거

전부 아니면 전무인 기준이 연속 기록을 취약하게 만듭니다. 90% 한 날에 0으로 리셋되면 그 기록은 지킬 가치가 없어집니다. 또 한 가지 중요한 규칙이 있습니다. **항목은 자신이 생성된 날짜 이후에 대해서만 계산됩니다.** 그렇지 않으면 오늘 루틴을 하나 추가하는 순간 과거 전체가 소급해서 실패한 날이 됩니다.

3.3 화면을 먼저 칠하고, 서버에는 그 다음에 묻는다

문제. 체크박스가 서버 응답을 기다린 뒤에야 상태를 바꾸고 있었습니다. 한국에서 Neon까지 왕복이 100-300ms인데 그동안 화면에 아무 변화가 없어, 애니메이션을 아무리 다듬어도 반응이 죽어 보였습니다.

결정. 클릭 즉시 로컬 상태로 칠하고, 서버 응답이 다르면 정정합니다. 되돌림 기준은 타이머가 아니라 서버가 준 새 값입니다.

app/components/use-task-actions.ts

```
// 새로 렌더된 서버 값이 마지막으로 본 값과 다르면 로컬 추측을 버린다.
// 타이머로 하면 느린 통신에서 요청 도중 만료돼 옛 상태가 번쩍인다.
const [seenServerDone, setSeenServerDone] = useState(serverDone);
if (seenServerDone !== serverDone) {
  setSeenServerDone(serverDone);
  setOptimistic(null);
}
const done = optimistic ?? serverDone;
```

검증 시 완료 API에 1500ms 지연을 걸고 클릭했을 때 체크박스는 **136ms**에 반응했습니다. 즉 네트워크를 기다리지 않습니다.

3.4 인증에 외부 의존성을 쓰지 않는다

결정. 비밀번호 해싱은 Node 내장 `scrypt` (OWASP 권장 KDF), 세션은 자체 `Session` 테이블의 불투명 토큰입니다. 새 패키지도, 서명 키 환경변수도 없습니다.

선택	이유
JWT가 아닌 DB 세션	즉시 무효화가 가능하고, 서명 키를 설정·보관·유출 걱정할 필요가 없음. 페이지가 이미 매 요청 DB를 조회하므로 추가 비용이 사실상 없음
쿠키엔 원본, DB엔 SHA-256	DB 덤프가 유출돼도 세션을 재사용할 수 없음
비밀번호 NFKC 정규화	한글은 조합형·분해형 두 형태가 있고 macOS와 Windows가 서로 다른 것을 만들. 정규화하지 않으면 같은 비밀번호로 다른 기기에서 로그인 시 실패
실패 메시지 단일화	없는 이메일과 틀린 비밀번호가 완전히 같은 문구·같은 소요 시간. 다르면 가입 여부를 알아내는 도구가 됨

3.5 권한 검사를 쿼리 안에 넣는다

문제. 계정이 생기기 전 API는 `id` 만 받으면 그대로 수정·삭제했습니다. 단일 사용자일 때는 무해했지만, 계정이 생기는 순간 `id`만 추측하면 남의 할 일을 지울 수 있는 IDOR 취약점이 됩니다.

app/api/tasks/[id]/route.ts

```
// update가 아니라 updateMany: 유일키가 아닌 조건을 받을 수 있어
// 소유권 검사가 같은 문장 안에서 일어난다.
const result = await prisma.task.updateMany({
  where: { id, userId: user.id },
  data: { /* ... */ },
});
if (result.count === 0) {
  // 없는 것과 남의 것을 같은 404로 답해 id 존재 여부를 노출하지 않는다.
  return NextResponse.json({ error: "not found" }, { status: 404 });
}
```

새 API를 추가할 때 반드시

조회는 `where` 에 `userId` 를 넣고, 수정·삭제는 `updateMany` / `deleteMany` 로 소유권을 조건에 포함시키십시오. `update({ where: { id } })` 는 편해 보이지만 그 자체로 취약점입니다. `TaskCompletion` 처럼 소유자 컬럼이 없는 테이블은 부모 `Task`의 소유권을 먼저 확인해야 합니다.

3.6 기존 데이터는 주인 없음으로 두고 첫 가입자가 인수

계정 도입 전에 만들어진 8건은 소유자가 없습니다. `Task.userId` 를 `nullable`로 두어, 주인 없는 행은 누구의 필터에도 걸리지 않아 노출되는 대신 보이지 않게 했습니다.

app/actions/auth.ts

```
// 계정이 하나도 없을 때만 인수 - 나중에 가입한 사람이
// 남의 데이터를 가져가는 일이 절대 불가능하도록.
const isFirstAccount = (await tx.user.count()) === 0;
const created = await tx.user.create({ data: { email, passwordHash, name } });
if (isFirstAccount) {
  await tx.task.updateMany({ where: { userId: null }, data: { userId: created.id } });
}
```

현재 상태 (2026-08-19 기준)

계정이 아직 0개이고 8건이 주인 없는 상태입니다. 즉 가장 먼저 가입하는 사람이 이 데이터를 가져갑니다. 제작자 본인이 먼저 가입해야 합니다. 인수가 끝나면 `userId` 를 `NOT NULL` 로 조이는 후속 마이그레이션을 권합니다.

3.7 가장 많은 분류에 가장 조용한 색

색은 Radix Colors 12단계 스케일을 씁니다. 배정은 빈도의 역순입니다.

분류	색	이유
정기 루틴	bronze (차분한 갈색)	가장 수가 많음. 채도가 높으면 주간 격자가 형광펜처럼 됨
날짜 있는 할 일	indigo	실제로 주의가 필요한 약속. 가장 눈에 띄어야 함
언젠가 할 일	teal + 점선 테두리	아직 확정이 아님을 점선으로 표현

분류는 색만이 아니라 **아이콘과 좌측 색 레일**로도 구분됩니다. 색각 이상 사용자를 고려한 것이며, 초기의 대각선 빗금 패턴은 칩 크기에서 뭉개져 폐기했습니다.

3.8 누름은 크기가 아니라 깊이로 표현

세 상태를 씁니다. 기본(바닥) → 호버(2px 떠오름 + 넓은 그림자) → 누름(기본보다 아래로 + 그림자 붕괴). 호버와 누름 사이의 낙차가 눌렀다는 감각을 만듭니다.

타이밍은 의도적으로 비대칭입니다. 호버 180ms(튀김 없음), 누름 100ms, **놓을 때만** 350ms 스프링. 양쪽을 같게 하면 누를 때 굵고 땔 때 기계적으로 느껴집니다. 크기 변화는 표면이 클수록 작게 주어 *체감 이동량*을 일정하게 맞췄습니다(`.press-soft` 2.2%, `.press-deep` 14%).

조절 지점

전부 `app/globals.css` 한 곳에 있습니다. `--ease-smooth`, `--ease-spring`, `.pressable`, `.lift`, `--shadow-rest` / `raised` / `pressed`. `prefers-reduced-motion` 블록도 같은 파일 하단에 있으니 새 애니메이션을 추가하면 거기에도 등록하십시오.

3.9 공휴일은 저장하지 않고 계산한다

광복절·추석 같은 공휴일을 `Task` 행으로 만들지 않았습니다. 만들었다면 체크와 삭제가 가능해지고, XP와 연속 기록에 섞여 들어가며, 계정이 하나 늘 때마다 같은 날짜가 통째로 복제됩니다. 대신 `lib/holidays.ts`는 **연도만 주면 결과가 나오는 순수 함수**이고 데이터베이스를 전혀 쓰지 않습니다. 과거·미래 어느 연도든 마이그레이션 없이 바로 나옵니다.

양력 공휴일은 규칙으로 계산합니다. 음력에서 오는 셋(설날·부처님오신날·추석)만 **2024-2030년 표**로 두었습니다. 정확한 음력→양력 변환에는 한국천문 연구원이 발표하는 삭(朔) 시각 자료가 필요한데, 직접 구현하면 코드가 몇 배로 늘고 어느 해 하루가 틀려도 알아차릴 방법이 없습니다. 표는 틀리면 눈에 보입니다.

대체공휴일은 「관공서의 공휴일에 관한 규정」 제3조를 그대로 옮겼습니다. 대부분은 **토·일과 겹칠 때**, 설·추석 연휴는 **일요일과 겹칠 때만**, 어린이날은 여기에 더해 **다른 공휴일과 겹칠 때도** 생깁니다. 일요일은 제2조가 그 자체를 공휴일로 정하고 있어 대체휴일이 될 수 없으며, 그래서 2027년 한글날(토 10/9)의 대체휴일은 일요일 10/10이 아니라 월요일 10/11입니다.

되돌리면 깨지는 것

표를 지우고 규칙만 남기면 **설날과 추석이 통째로 사라집니다**. 반대로 공휴일을 `Task`로 옮기면 공휴일 덕분에 연속 기록이 올라가고, 사용자가 공휴일을 삭제할 수 있게 됩니다.

3.10 드래그는 포인터 이벤트로, 배치 계산은 브라우저에서

할 일을 끌어 옮기는 기능은 HTML5 드래그 앤 드롭 API를 쓰지 않습니다. 그 API는 **터치 화면에서 아예 동작하지 않습니다**. 이 앱은 노트북만큼이나 휴대폰에서 쓰이므로, 마우스·터치·펜을 한 경로로 처리하는 Pointer Events 위에 직접 만들었습니다.

놓을 수 있는 자리는 포인터 아래를 히트 테스트해 `[data-drop]` 조상을 찾는 방식으로 정합니다. 새 영역은 데이터 속성 두 개만 붙이면 참여하며 등록도 ref도 필요 없습니다.

어느 블록이 어디 놓이는지 계산하던 코드가 `app/week/page.tsx`에서 `app/components/week-board.tsx`로 옮겨졌습니다. **드롭은 서버가 알기 전에 화면에 반영돼야 하기 때문**이며, 3.3과 같은 이유입니다. 이제 페이지는 조회와 직렬화만 하고 배치는 브라우저가 합니다.

정기 루틴은 **같은 요일 안에서 시간만** 옮겨집니다. 루틴은 날짜가 아니라 요일 집합으로 정의되므로, 목요일 칸에 놓는 순간 반복 규칙 전체를 다시 써야 하고 월·수·금 루틴이 조용히 목요일 하루짜리가 됩니다. 그래서 다른 요일 칸은 아예 받지 않습니다.

지우면 즉시 깨지는 두 가지

6px 이동 문턱 — 없으면 모든 탭이 길이 0짜리 드래그로 잡혀 체크박스가 토글되지 않습니다. **드래그 직후 클릭 한 번 삼키기** — 없으면 월간에서 칩을 놓는 순간 칩이 들어 있던 링크를 따라가고, 시간표에서는 방금 옮긴 할 일이 완료 처리됩니다.

3.11 촉진 메일은 설정하기 전까지 아무 일도 하지 않는다

며칠 활동이 없으면 메일을 보냅니다. 본문이 "돌아오세요"로 시작하지 않고 **기한이 지난 할 일의 제목**부터 보여줍니다. 앞의 문장은 스팸과 구분이 안 되지만, 밀린 항목 두 개를 이름으로 부르는 메일은 앱을 열 이유가 됩니다.

메일 발송은 Resend의 REST API를 `fetch` 로 직접 부릅니다. SDK를 쓰지 않아 의존성이 늘지 않으며, `lib/auth.ts` 가 인증 라이브러리를 쓰지 않는 것과 같은 이유입니다. `lib/mailer.ts` 는 **예외를 던지지 않습니다**. 메일 업체는 어떤 기능에서든 가장 자주 고장나는 부분이라, 모든 결과를 값으로 돌려주어 업체 장애가 작업 전체를 끌어내리지 못하게 했습니다.

도메인이 없으면 Resend 공용 주소로 나가는데, 이 경우 두 가지 제약이 있습니다. **가입한 본인 주소로만** 보낼 수 있고, Gmail이 스팸으로 분류할 가능성이 높습니다(7장). 보내는 주소를 `MAIL_FROM` 환경변수로 빼둔 것은 도메인이 생겼을 때 코드를 건드리지 않기 위해서입니다.

키가 없으면 기능 전체가 **조용히 쉽니다**. `RESEND_API_KEY` 가 없으면 "미설정"이라고 보고하고 아무것도 보내지 않으며, `CRON_SECRET` 이 없으면 엔드포인트가 503으로 스스로를 막습니다. 설정 전에 배포해도 아무 일도 일어나지 않는다는 뜻입니다.

조용한 기간은 **서울 달력 날짜**로 셉니다. "3일째 안 들어왔다"는 달력에 대한 진술이고, 경과 시간으로 재면 마지막 방문이 몇 시였느냐에 따라 답이 달라집니다.

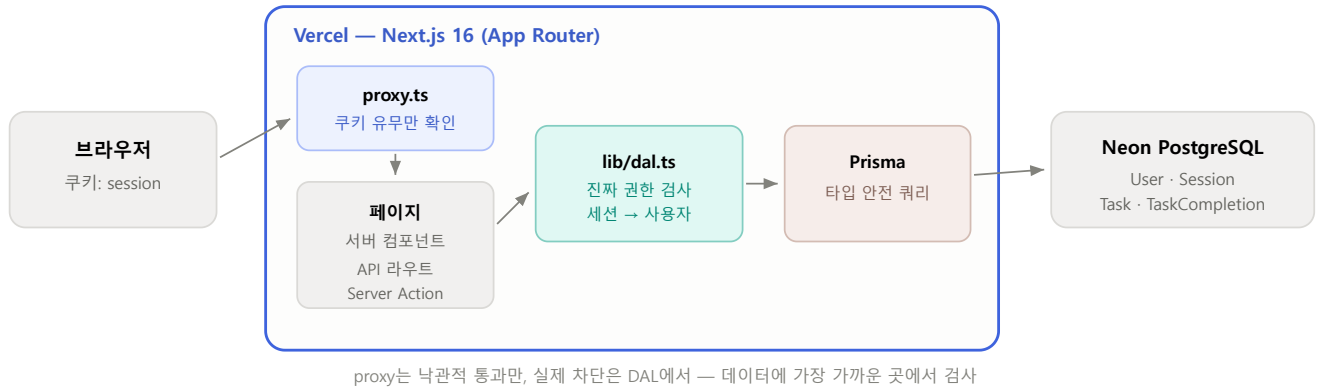
`lastActiveAt` 은 데이터 접근 계층에서 `after()` 로 기록합니다. 응답이 나간 뒤에 실행되므로 사용자가 기다리는 페이지에 지연을 더하지 않습니다. 한 시간에 한 번으로 제한하는데, 없으면 모든 페이지 조회와 프리페치마다 쓰기가 발생합니다.

수신 거부 버튼 뒤에 있는 이유

메일 속 링크는 GET이지만 **제안만 합니다**. 실제 전환은 버튼 뒤의 POST가 합니다. 메일 스캐너와 링크 미리보기는 본문의 모든 URL을 열어보기 때문에, GET이 곧바로 작동하면 **독자 대신 스캐너가 수신 거부를 눌러버립니다**. 다시 켜는 화면이 앱 어디에도 없으므로 되돌릴 방법도 없었을 것입니다. 링크 서명에는 그 계정의 비밀번호 해시를 키로 씁니다. 새 비밀번호를 만들 필요가 없고, 비밀번호를 바꾸면 옛 링크가 자동으로 죽습니다.

4 시스템 구조

4.1 전체 그림



요청 경로. 권한 판단은 화면이 아니라 데이터 접근 지점에 있다.

4.2 데이터 모델

테이블	핵심 컬럼	비고
User	email (유일), passwordHash, name	이메일은 소문자·공백제거 후 저장. 대소문자만 다른 중복 가입 방지
Session	tokenHash (유일), userId, expiresAt	쿠키 값의 SHA-256만 보관. 만료는 서버에서 판단
Task	title, dueDate, weekdays, startTime / endTime, priority, archived, userId	세 분류를 모두 담음. userId는 nullable(3.6 참조)
TaskCompletion	taskId, date, completedAt	(taskId, date) 유일 제약이 중복 완료를 DB 차원에서 차단

User 삭제 시 Task와 Session이, Task 삭제 시 TaskCompletion이 **연쇄 삭제(cascade)**됩니다. 계정을 지우면 그 사람의 데이터가 전부 사라진다는 뜻이므로 운영 시 주의가 필요합니다.

시간 컬럼의 두 가지 성격

dueDate와 TaskCompletion.date는 **날짜**이며 항상 서울 날짜의 UTC 자정으로 저장됩니다. 반면 startTime · endTime은 **날짜가 없는 벽시계 문자열**("09:30")입니다. 시간대 개념이 없으므로 시간대 변환을 적용하면 안 됩니다.

4.3 코드 배치 원칙

위치	담는 것
<code>lib/task-utils.ts</code>	순수 날짜·시간·격자 계산. Prisma 타입을 모름 → 단독 테스트가 쉬움
<code>lib/gamification.ts</code>	연속 기록·XP·배지. 전부 기존 완료 기록에서 파생되며 별도 테이블 없음
<code>lib/auth.ts</code>	비밀번호 해싱, 세션 발급·파기
<code>lib/dal.ts</code>	세션 쿠키 → 사용자. 모든 권한 판단의 단일 출입구
<code>app/*/page.tsx</code>	서버 컴포넌트. DB 조회 후 화면 조립. 단 주간·월간은 조회와 직렬화까지만 하고 배치는 보드에 넘김
<code>app/components/week-board.tsx</code> <code>month-board.tsx</code>	달력 배치 계산과 낙관적 이동. 드롭이 서버보다 먼저 화면에 반영돼야 해서 클라이언트에 있음(3.10)
<code>app/components/drag-context.tsx</code>	포인터 기반 드래그 엔진. 특정 화면에 매여 있지 않음
<code>app/components/*</code>	화면 조각. 상호작용이 있으면 <code>"use client"</code>
<code>app/api/*</code>	브라우저가 <code>fetch</code> 로 호출하는 JSON 엔드포인트
<code>app/actions/*</code>	Server Action. 폼 제출을 서버에서 직접 처리

게임화 데이터를 저장하지 않는 이유

연속 기록·XP·배지는 모두 `TaskCompletion` 에서 그때그때 계산합니다. 저장했다면 기준을 바꿀 때마다(예: 70% → 60%) 과거 데이터를 다시 계산하는 마이그레이션이 필요했을 것입니다. 지금은 상수 하나만 바꾸면 과거까지 일관되게 반영됩니다. 사용자가 수천 명이 되면 재검토할 지점입니다.

5 인증과 권한

5.1 로그인 흐름

- 1 가입/로그인 폼 제출 — `app/actions/auth.ts` 의 Server Action이 서버에서 직접 실행됩니다. 비밀번호가 네트워크 위 JSON으로 떠다니지 않습니다.
- 2 비밀번호 검증 — `scrypt` 로 해시 비교 (`timingSafeEqual` 사용).
- 3 세션 발급 — 32바이트 난수 토큰 생성 → SHA-256을 DB에 저장 → 원본을 `httpOnly · sameSite=Lax · 30일` 쿠키로 내려줌.
- 4 이후 모든 요청 — `proxy.ts` 가 쿠키 유무만 보고, `lib/dal.ts` 가 DB에서 실제 유효성과 만료를 확인합니다.

proxy가 로그인한 사용자를 /login에서 쫓아내지 않는 이유

만료된 쿠키는 존재하지만 유효하지 않습니다. proxy가 쿠키만 보고 "로그인됨"으로 판단해 `/` 로 보내면, `/` 의 진짜 검사가 실패해 다시 `/login` 으로 보내고, 무한 반복이 됩니다. 그래서 그 리디렉션은 DB를 실제로 조회하는 로그인 페이지 자신이 수행합니다.

5.2 검증된 사항

계정 기능은 다음 항목을 실제 서버 대상으로 확인했습니다.

항목	결과
비로그인 상태로 <code>/</code> , <code>/week</code> 접근	307 → <code>/login</code>
비로그인 상태로 <code>/api/tasks</code> 호출	401
B 계정이 A 계정의 할 일 수정·삭제·완료 시도	모두 404, A의 데이터 불변
각 계정의 목록 조회	자기 것만 보임. 주인 없는 행은 양쪽 모두 안 보임
만료 토큰 / 위조 토큰	모두 거부
세션 쿠키	<code>httpOnly</code> , <code>sameSite=Lax</code> , <code>document.cookie</code> 에 미노출
없는 이메일 vs 틀린 비밀번호	문구·소요시간 동일
첫 가입자의 기존 데이터 인수	롤백되는 트랜잭션 안에서 8건 전량 인수 확인

6 운영

6.1 배포

`main` 브랜치에 푸시하면 Vercel이 자동 배포합니다. 평균 1-2분. 빌드 명령은 `prisma generate && next build` 입니다.

마이그레이션은 빌드에 포함되어 있지 않습니다

초기에 빌드 단계에서 `prisma migrate deploy` 를 실행했다가 배포가 두 번 실패했습니다(빌드 환경에 DB 접속 정보가 없는 상태). 지금은 **스키마 변경 시 사람이 직접** 마이그레이션을 적용해야 합니다. 6.3을 따르십시오.

6.2 환경변수

이름	용도
<code>DATABASE_URL</code>	런타임용 풀링 연결. Vercel Storage에서 Neon을 추가하면 자동 설정
<code>DATABASE_URL_UNPOOLED</code>	마이그레이션 전용 직접 연결. <code>prisma migrate</code> 는 세션 단위 연결이 필요해 풀러로는 동작하지 않음
<code>CRON_SECRET</code>	Vercel 크론이 <code>/api/cron/nudge</code> 를 부를 때 붙이는 인증값. 없으면 엔드포인트가 503으로 스스로를 막음
<code>RESEND_API_KEY</code>	메일 발송 키. 없으면 "미설정"이라 보고하고 보내지 않음
<code>MAIL_FROM</code>	보내는 사람. 도메인이 없으면 비워둠(Resend 공용 주소 사용)
<code>NUDGE_AFTER_DAYS</code>	며칠 조용하면 보낼지. 기본 3. 다음 발송까지의 간격도 같은 값
<code>APP_URL</code>	메일 속 링크가 가리킬 주소. Vercel에서는 자동 판별하므로 보통 비워둠

로컬 `.env` 는 dev 브랜치만 봅니다

`DATABASE_URL` 과 `DATABASE_URL_UNPOOLED` 에는 **Neon dev 브랜치** 주소만 넣으십시오. 운영 주소는 `PROD_` 접두사를 붙여 따로 보관합니다. Prisma가 읽는 이름은 저 둘뿐이라, 이름이 다르면 어떤 prisma 명령도 — 실수로 shadow database에 지정하더라도 — 운영에 닿을 수 없습니다. 8.2의 사고가 이 분리를 만들었습니다.

Vercel 환경변수는 빌드 시점에 고정됩니다

값을 바꾼 뒤 **반드시 재배포**해야 반영됩니다. 과거에 Neon 연동 시 접두사 설정을 잘못해 `DATABASE_URL_PG` `USER` 같은 변수가 생기고, 동시에 빈 `DATABASE_URL` 이 존재해 런타임 오류가 난 적이 있습니다. 값이 비어 있지 않은지, 이름이 정확한지 확인하십시오.

6.3 스키마 변경 절차

운영 DB에 대해 `prisma migrate dev` 를 실행하지 마십시오. 스키마 불일치가 감지되면 **데이터베이스 초기화**를 제안하며, 잘못 승인하면 전체 데이터가 사라집니다. 아래 절차를 쓰십시오.

- 1 `prisma/schema.prisma` 를 수정합니다.
- 2 적용하지 않고 SQL만 생성해 **내용을 눈으로 확인**합니다. `DROP` · `DELETE` · `TRUNCATE` 가 있으면 멈추십시오.

```
npx prisma migrate diff \
  --from-url "$DATABASE_URL_UNPOOLED" \
  --to-schema-datamodel prisma/schema.prisma --script
```

- 3 확인한 SQL을 `prisma/migrations/<타임스탬프>_<이름>/migration.sql` 로 저장합니다.
- 4 `npx prisma migrate deploy` 로 적용합니다.
- 5 `npx prisma generate` 로 타입을 갱신하고 커밋합니다.

--shadow-database-url 에 실제 DB 주소를 넣지 마십시오

위 2단계가 `--from-url` 을 쓰는 데에는 이유가 있습니다. 그것은 데이터베이스를 **읽기만** 합니다. 반면 `--from-migrations` 는 마이그레이션을 재생할 shadow 데이터베이스를 요구하고, **Prisma는 shadow 데이터베이스를 초기화합니다**. 거기에 운영 주소를 넣으면 테이블이 전부 지워집니다.

2026-08-20에 정확히 그 일이 일어났습니다(8.2). 이 절차서에는 그때도 `--from-url` 이라고 적혀 있었습니다. **문서가 틀린 것이 아니라 지키지 않은 것입니다.**

기존 행이 있는 테이블에 **필수 컬럼**을 추가할 때는 곧바로 `NOT NULL` 로 만들 수 없습니다. ① nullable로 추가 → ② 기존 행 채우기 → ③ `NOT NULL` 로 조이기, 세 단계로 나누십시오.

6.4 자주 쓰는 명령

명령	용도
<code>npm run dev</code>	개발 서버 (localhost:3000)
<code>npx tsc --noEmit</code>	타입 검사

<code>npx eslint .</code>	린트
<code>npx next build</code>	배포 전 최종 확인 — 푸시 전 반드시 실행
<code>npx prisma studio</code>	DB 내용을 브라우저에서 직접 확인·수정
<code>npx prisma migrate deploy</code>	마이그레이션 적용

7 알려진 한계와 리스크

인수인계에서 가장 중요한 장입니다. 아래 항목은 전부 **알고도 남겨둔** 것이며, 모르고 지나간 것이 아닙니다.

항목	심각도	내용과 영향
이력 보관이 6시간뿐	중간	Neon 무료 플랜의 시점 복원 가능 구간이 6시간 입니다. 8.2의 사고는 15분 만에 발견해 되살렸지만, 같은 실수를 밤에 저질렀다면 아침에는 복구가 불가능했습니다. 데이터가 쌓일수록 유료 플랜이나 정기 덤프를 검토해야 합니다.
운영 주소가 로컬 <code>.env</code> 에 남아 있음	낮음	2026-08-20에 Neon <code>dev</code> 브랜치를 분리해 로컬은 그쪽만 봅니다. 운영 주소는 같은 파일에 <code>PROD_</code> 접두사로 남아 있는데, Prisma가 읽지 않는 이름이라 닿지 않습니다. 이 이름을 <code>DATABASE_URL</code>로 되돌리지 마십시오. 되돌리는 순간 8.1과 8.2가 다시 가능해집니다.
촉진 메일이 스팸함으로 감	낮음	도메인이 없어 Resend 공용 발신 주소(<code>onboarding@resend.dev</code>)를 씁니다. 수많은 사용자가 시험용으로 공유하는 주소라 Gmail이 이미 나쁜 평판을 학습해 두었고, 2026-08-20 첫 발송이 실제로 스팸함으로 갔습니다. 내용이나 조판 문제가 아닙니다. 개인용으로는 Gmail 필터("스팸으로 보내지 않음")면 충분합니다. 근본 해결은 도메인을 사서 Resend에 인증하고 <code>MAIL_FROM</code> 만 바꾸는 것이며, 그 한 줄로 끝나도록 환경변수로 빼 두었습니다. 본인 외의 사람에게 보내려면 어차피 필요합니다.
환경변수는 재배포해야 반영됨	낮음	Vercel에서 값만 바꾸고 재배포하지 않으면 실행 중인 배포는 옛 값을 계속 씁니다. <code>NUDGE_AFTER_DAYS</code> 를 3으로 되돌렸는데도 크론 응답이 <code>"afterDays":1</code> 이면 이것입니다. 응답에 그 값이 찍히도록 해 둔 이유가 여기에 있습니다.
삭제 되돌리기 없음	높음	X 버튼은 즉시 영구 삭제 입니다. 휴지통도 확인창도 없습니다. 스키마에 <code>archived</code> 컬럼이 이미 있으나 삭제에 쓰이지 않습니다. 삭제를 <code>archived = true</code> 로 바꾸는 것만으로 대부분의 사고를 막을 수 있습니다.
비밀번호 재설정 없음	높음	메일 발송 수단이 없어 구현하지 않았습니다. 사용자가 비밀번호를 잊으면 운영자가 DB에서 직접 해시를 교체 해야 합니다. 외부에 배포하기 전 반드시 해결해야 할 항목입니다.
로그인 시도 제한 없음	중간	무차별 대입 방어가 <code>sCrypt</code> 의 계산 비용뿐입니다. 사용자가 늘면 IP·계정 단위 제한이 필요합니다.
할 일 수정 기능 없음	중간	만들기·완료·날짜배치·삭제만 있습니다. 제목이나 시간을 고치려면 지우고 다시 만들어야 하며, 그 과정에서 완료 기록이 사라집니다.

자동 테스트 없음	중간	검증은 매번 일회용 스크립트로 수행하고 폐기했습니다. 회귀를 잡아줄 상시 테스트가 없어, 날짜 로직처럼 조용히 깨지는 부분이 위험합니다.
모바일 앱·알림 없음	중간	웹만 있습니다. 리텐션 장치가 연속 기록 하나뿐입니다.
정기 루틴 0건	낮음	현재 등록된 루틴이 없어 연속 기록과 연간 히트맵이 사실상 비어 있습니다. 기능 결함이 아니라 데이터 부재입니다.
음력 공휴일 2030년까지	낮음	설날·부처님오신날·추석은 <code>lib/holidays.ts</code> 의 표에 2024-2030년만 들어 있습니다. 2031년 이후를 열면 양력 공휴일만 나오며, 그 사실이 월간 화면 하단에 표시되므로 조용히 틀리지는 않습니다. 표에 연도를 추가하면 해결됩니다.
드래그를 대신할 키보드 경로 없음	중간	날짜 있는 할 일을 다른 날·시간으로 옮기는 방법이 드래그뿐 입니다. 미배치함 항목만 "이 날로 옮기기" 버튼으로도 옮길 수 있습니다. 할 일 수정 기능이 생기면 자연스럽게 해결됩니다.
잘못 놓아도 되돌리기 없음	낮음	드롭은 즉시 저장되고 취소가 없습니다. 다시 끌어 돌려놓을 수는 있지만 원래 시각을 기억하고 있어야 합니다.
월간에서 4번째 이후 칩은 못 끈다	낮음	한 칸에 칩을 3개까지만 그리고 나머지는 "+N건 더"로 접습니다. 접힌 항목은 화면에 없으므로 끌 수도 없습니다. 주간 화면에서는 전부 보입니다.
세션 자동 연장 없음	낮음	30일 고정입니다. 만료되면 다시 로그인해야 합니다.
만료 세션 정리 없음	낮음	만료된 <code>Session</code> 행이 계속 쌓입니다. 조회 시 만료를 확인하므로 보안 문제는 아니고 용량 문제입니다.

8 사고 기록

두 건 모두 같은 뿌리에서 나왔습니다. 개발 작업이 운영 데이터베이스를 직접 보고 있었다는 것입니다. 인수인계 문서에서 사고 기록은 기능 설명보다 오래 쓸모가 있습니다.

8.1 할 일 1건 손실 (2026-08-18)

발생	2026-08-18, UI 모션 작업 검증 중
증상	할 일 8건 중 1건(2차 합격자 부서 배치 확정)이 사라짐
발견	로그인 작업 중 마이그레이션 후 건수를 확인하다가 7건인 것을 발견
원인	브라우저 자동화 검증을 운영 데이터베이스에 연결된 개발 서버에 대고 실행. 좌표 기반 마우스 조작이 의도치 않은 요소를 눌렀음
복구	원본이 Notion에 있어 복원. 생성 시각 간격이 다른 항목은 0.21초인데 해당 위치만 0.52초였던 점으로 누락 지점을 특정

왜 막지 못했나. 같은 세션에서 이미 한 번 경고 신호가 있었습니다. 검증 중 생성된 완료 기록 2건이 운영 DB에 들어간 것을 발견하고 삭제했지만, 그때 증상만 치우고 원인(테스트가 운영 DB를 본다)은 고치지 않았습니다. 그 결과 다음 검증에서 더 큰 손실이 발생했습니다.

이 사고에서 가져갈 원칙

- 운영 데이터에 대고 자동화 테스트를 돌리지 말 것. 이것이 유일한 근본 원인입니다.
- 좌표 기반 마우스 조작 대신 이름·역할 기반 선택자를 쓸 것. 레이아웃이 조금만 움직여도 엉뚱한 것을 누릅니다.
- 파괴적 작업 전후로 건수를 확인할 것.
- 같은 종류의 이상 징후가 두 번째 나타나면 증상이 아니라 원인을 고칠 것.

8.2 데이터베이스 전체 초기화 (2026-08-20)

발생	2026-08-20 약 19:00 KST, 촉진 메일 기능의 마이그레이션 SQL을 미리 보려던 중
증상	계정 1개, 할 일 8건, 세션이 전부 사라짐. 테이블 구조와 <code>_prisma_migrations</code> 이력까지 함께 초기화됨
원인	<code>prisma migrate diff --from-migrations ... --shadow-database-url "<운영 주소>" . --from-migrations</code> 는 마이그레이션을 재생할 shadow 데이터베이스를 요구하고, Prisma는 shadow 데이터베이스를 초기화합니다.

발견	약 15분 뒤. 이어서 실행한 <code>migrate deploy</code> 가 <code>P3005</code> 로 실패해 원인을 캐다가 확인
복구	Neon 시점 복원으로 <code>2026-08-20T09:50:00Z</code> 상태로 되돌림. 0.39초 소요, 전량 복구

왜 막지 못했나. 이 문서 6.3에는 그때도 `--from-url` 을 쓰라고 적혀 있었습니다. **절차서가 틀린 것이 아니라 지키지 않은 것입니다.** 더 나쁜 것은, 같은 세션에서 불과 몇 분 전에 운영 DB를 통째로 지우는 `db:seed` 스크립트를 "위험하다"며 삭제했고 개발용 DB 분리를 세 번 권고했다는 점입니다. **권고와 행동이 어긋났습니다.** 8.1의 근본 원인을 그대로 둔 채 다른 작업을 계속한 결과이기도 합니다.

이 사고에서 가져갈 원칙

- `--shadow-database-url` 에 실제 데이터베이스 주소를 절대 넣지 말 것. shadow 데이터베이스는 초기화됩니다.
- 스키마 비교는 읽기 전용인 `--from-url` 만 쓸 것(6.3).
- 위험한 이름은 **손이 닿지 않는 곳에 둘 것.** 운영 주소를 `PROD_` 접두사로 옮긴 것이 이 사고의 실제 대책입니다. 규율이 아니라 구조로 막아야 합니다.
- 복구 창이 6시간뿐이라는 것을 기억할 것. 발견이 늦으면 절차를 알아도 소용이 없습니다.

같은 날 Neon `dev` 브랜치를 분리해 로컬이 운영을 볼 수 없게 했습니다. 8.1의 기록이 "개발용 DB 분리가 여전히 필요합니다"로 끝났었는데, **그 문장을 이틀간 방치한 대가가 8.2였습니다.**

9 다음에 할 일

우선순위 순입니다. 위 세 가지는 성격상 기능 추가가 아니라 사고 예방입니다.

즉시

- 1 **제작자 본인 계정 가입.** 기존 8건이 첫 가입자에게 귀속됩니다. 아직 계정이 0개이므로 다른 사람이 먼저 가입하면 그 사람에게 갑니다.
- 2 **개발용 DB 분리.** Neon 대시보드에서 브랜치를 하나 만들고 로컬 `.env` 가 그것을 보게 합니다. 8장 사고의 근본 원인입니다.
- 3 **삭제를 보관 처리로 변경.** 이미 있는 `archived` 컬럼을 사용하면 화면에서만 사라지고 데이터는 남습니다.

외부에 공개하기 전

- 1 **비밀번호 재설정.** 메일 발송 수단(예: Resend) 연동이 필요합니다.
- 2 **로그인 시도 제한.**
- 3 **할 일 수정 기능.** 지우고 다시 만들면 완료 기록이 사라지므로 사용자 입장에서 손실이 큼니다.
- 4 **핵심 로직 자동 테스트.** 최소한 `lib/task-utils.ts` 와 `lib/gamification.ts` 는 순수 함수라 테스트가 쉽습니다.

제품을 키우려면

- 1 **과적 방지의 전면화.** 하루 용량이 넘칠 때 실제로 경고하거나 말리는 동작. 이 앱의 유일한 차별점이며 현재는 숫자로만 표시됩니다.
- 2 **미배치 항목 드래그 배치.** 지금은 클릭 방식입니다.
- 3 **모바일 대응(PWA)과 알림.**
- 4 **그룹 연속 기록.** 혼자 쓰는 기록은 이탈 비용이 없습니다. 함께하는 사람이 생기면 그때부터 방어 가능한 해자가 됩니다.

이 문서는 커밋 `20e1b7e` 기준입니다. 코드가 바뀌면 이 문서도 함께 갱신하십시오. 특히 3장(핵심 설계 결정)과 7장(한계)은 사실과 어긋나는 순간 문서 전체의 신뢰가 사라집니다. 코드와 다른 설명은 없는 것보다 나쁩니다.